

30分で完走

完全未経験歓迎

実践ハンズオン

GitHub入門講座

～ Gitの仕組みを理解して、初めての草を生やそう ～

Cypher-郭嘉宏 (AI & Data Science Community)

この30~60分のゴール

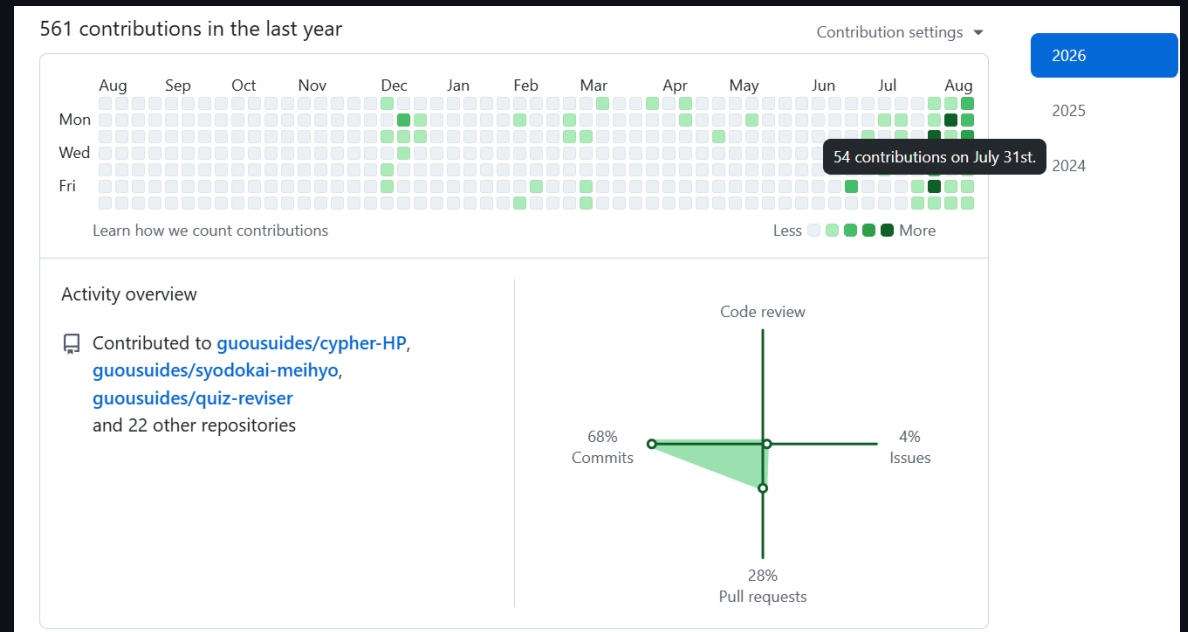
🎯 皆さんのPull Requestがmergeされ、GitHubプロフィールに「草」が生えている状態！

🌱 草を生やすとは？

GitHubで開発活動（コミット・PR等）をした日に、プロフィールに付く**緑のマス目（コントリビューション）**のこと。

📄 Pull Request（PR）とは？

「自分の変更を本番に取り込んでください！」という**チームへのお願い・提案**のこと。



▲ GitHubプロフィール画面（草が生えた状態）

スタート前の環境クイックチェック

本講座は **GitHubアカウント作成済み** の状態からスタートします。

手元のターミナル（Windowsは Git Bash / Macはターミナル）で動作確認しましょう！

#	確認項目	打つコマンド	期待される結果
1	Gitが入っている	<code>git --version</code>	<code>git version 2.xx.x</code>
2	名前が登録されている	<code>git config --global user.name</code>	あなたのGitHub ID
3	メールが登録されている	<code>git config --global user.email</code>	GitHubの登録メール
4	GitHubログイン済み	<code>gh auth status</code>	✓ Logged in to github.com

⚠ 未設定の項目がある人や赤字のエラーが出た人は**遠慮なく声をかけてください！**

第1章 (03-07分)

バージョン管理ってなに？

ファイルの歴史を記録する・タイムマシン機能

「最終版_本当の最終.docx」ってどれなん????

📁 こんなファイル名ありませんか？

- 企画書_最新.docx
- 企画書_最新_修正.docx
- 企画書_本当の最終版.docx
- 企画書_本当の最終版_修正2_田中.docx

😱 発生する悲劇

- どれが本当の最新版か分からない！
- 誰がどこを変更したか追えない！
- 過去のバージョンに戻せない！
- 上書き事故で大事なデータが消滅…

Git（バージョン管理）を使うとどうなる？

 **ファイル名は1つのまま！過去の全履歴をきれいに保存**

 **タイムマシン機能**

過去の任意のセーブポイント（コミット）にいつでも安全に戻せる！

 **1行単位の履歴**

「誰が・いつ・何を変更したか」がすべて完全に記録される！

 **同時並行の開発**

複数人で同じファイルを編集しても壊れない（ブランチ機能）！

第2章 (07-11分)

Gitの仕組みと3大用語

荷物の発送にたとえて一発理解！

覚える用語は3つだけ！

1. リポジトリ (Repository)

ファイルや変更履歴をしまっておく「**保存箱・貯蔵庫**」のこと。

- ・ **ローカルリポジトリ**：自分のPC内
- ・ **リモートリポジトリ**：GitHub（クラウド）

2. コミット (Commit)

ファイルの変更をリポジトリに「**記録・確定**」すること。

「何を直したか」のメッセージ（コミットメッセージ）を添えて記録します。

3. ワークツリー と ステージ

- ・ **ワークツリー（作業場）**：あなたが今実際にファイルを編集しているフォルダ。
- ・ **ステージ（控え室）**：コミットする前に「これだけ記録する」と選んだ変更を置く場所。

荷物の発送でたとえる「4領域モデル」

変更したファイルは、手元のPCからGitHub（クラウド）まで**4つの場所**を順番に移動します。

1 ワークツリー（作業場）

手元のPCでファイルを編集集中の状態（部屋の机の上）

2 ステージ

`git add` で送る変更をダンボールに詰めた状態

3 ローカルリポジトリ

`git commit` でガムテープで封をして手元に保存

4 リモートリポジトリ（GitHub）

`git push` でクラウド郵便局へ発送・共有！

よくある疑問：「なぜインデックス（add）があるの？」

「ファイルを編集したら、そのまま保存（commit）じゃダメなの？」

💡 理由：関係ない変更を混ぜずに綺麗に記録するため！

例えば、同時に2つのファイルを編集したとします：

- `login.py`（ログイン機能の追加） ➡ 今回のコミットに含めたい！
- `test_memo.txt`（個人的なメモ・落書き） ➡ 人に見せたくない…

`git add login.py` とすることで、「送りたい物だけ」を綺麗に選んで箱詰めできます。

Git と GitHub の違い

名前は似ていますが、役割がまったく違います！

	Git (ギット)	GitHub (ギットハブ)
正体	あなたのPCに入る アプリ・道具	ネット上の Webサービス
場所	手元のパソコンの中	クラウド (インターネット)
役割	ファイルの変更履歴を 記録 する	履歴を 共有 してみんなで開発する
ネット	不要 (オフラインで動く！)	必要

たとえ：Git = **カメラ (撮影する道具)** | GitHub = **Instagram (写真を共有する場所)**

第3章 (11-13分)

ターミナルの基本

黒い画面は「文字でおしゃべりする道具」

ターミナルで一番大事なこと：「今どこにいるか」

ターミナルは「今いる場所（カレントディレクトリ）」を基準にして動きます。

カレントディレクトリ（Current Directory）＝ ターミナルが今開いているフォルダのこと

```
/Users/taro/  
├─ Desktop/    ← デスクトップ  
├─ Downloads/  ← ダウンロード  
└─ Documents/  ← 書類
```

- 今 **Desktop** にいるのか？ それとも **Downloads** にいるのか？
- **場所を見失ったら、まず「今どこ？」を確認する**のが鉄則です！

覚えるコマンドはとりま3つだけ！

`pwd` (今どこ？)

Print **W**orking **D**irectory

今いるフォルダの場所（パス）を表示します。

`ls` (何がある？)

List

今いるフォルダの中にあるファイル一覧を表示します。

`cd フォルダ名` (そこへ移動！)

Change **D**irectory

指定したフォルダの中に移動します。(例： `cd Desktop` でデスクトップへ移動)

※ `cd ..` (ドット2つ) で「1つ上の階層に戻る」

第4章 (13-24分)

ハンズオン本編！

自己紹介ファイルを作って Pull Request を出そう

これからやるハンズオンの流れ（GitHub Flow）

実務のエンジニアが毎日やっている「王道の開発手順」を体験します！

1. 📄 **clone** ➡ プロジェクトを自分のPCに持ってくる
2. 🌿 **branch** ➡ 自分専用の作業ブランチ（枝）を作る
3. 📝 **編集** ➡ `members/あなたのID.md` を作成して自己紹介を書く
4. 📦 **add** ➡ 変更をインデックス（発送箱）に入れる
5. 🗑️ **commit** ➡ 変更を手元の記録として確定する
6. 🚀 **push** ➡ GitHubに送り届ける
7. 📧 **PR作成** ➡ 「取り込んでください！」とPRを出す
8. 🎉 **Merge** ➡ レビュー・承認されて合体！

Step 1. リポジトリを手元に持ってくる

CLI **Desktop** に移動して、リポジトリを丸ごと複製（clone）します。

```
cd Desktop  
git clone https://github.com/guousuideo/git-lec.git
```

VS Code 「クローン」 ボタンを押し、上のURLを貼り付けます。

期待される結果： デスクトップに **git-lec** フォルダが新しく作られます！

```
cd git-lec
```

⚠ 超重要： 必ず **cd git-lec** でフォルダの中に入ってください！

Step 2. 自分専用の作業ブランチを作る

みんなで同じ本番（`main`）を直接いじると事故が起きます。

安全のため「自分専用の枝分かれ（ブランチ）」を作って作業します。

CLI 自分のID名でブランチを作成して切り替えます：

```
git switch -c add-member-あなたのID
```

(例： `git switch -c add-member-taro-yamada`)

VS Code 左下の `main` をクリック ➡ 「新しい分岐の作成」

期待される結果： `Switched to a new branch 'add-member-...'` と出ればOK！

Step 3. 自己紹介ファイルを作る

`members/` フォルダの中に、`あなたのGitHub ID.md` を作成します！

📁 ファイルの場所

`members/taro-yamada.md`

(※拡張子は `.md` です)

📝 中に書く内容（自由）

自己紹介

- 名前: 山田 太郎
- 所属: 〇〇大学 / Cypher
- 一言: GitHubマスターになります！

💡 VS Codeのエクスプローラーから `members` フォルダを右クリックして「新規ファイル」で作ると簡単です！

Step 4. 今の状態を確認する

今、Gitから見てファイルがどうなっているか確認してみましょう！

```
git status
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  members/taro-yamada.md   # ← 赤文字で表示されます！
```

- **赤文字 (Untracked)** = 「部屋に新しい物があるけど、まだ発送箱（ステージ）には入っていませんよ」という合図です。

Step 5. 送信箱に入れる (add)

作成した自己紹介ファイルを、インデックス（送信箱）に乗せます。

CLI

```
git add members/あなたのID.md
```

(または `git add .` で今のフォルダの全変更をステージ)

VS Code ソース管理パネルで、ファイル名の横の「+」ボタンをクリック

もう一度 `git status` を打ってみましょう：

```
Changes to be committed:
  new file:   members/taro-yamada.md  # ← 緑文字に変わった！
```

Step 6. 記録を確定して封をする (commit)

「何を変更したか」のメッセージを添えて、手元に記録します。

CLI

```
git commit -m "docs: add taro-yamada to members"
```

VS Code メッセージ欄に `docs: add ...` と入力して **「コミット」** ボタン

```
[add-member-taro-yamada a1b2c3d] docs: add taro-yamada to members
1 file changed, 5 insertions(+)
create mode 100644 members/taro-yamada.md
```

これで「PC内の保存箱」にしっかり記録されました！ 🎉

Step 7. GitHubへ送り出す（push）

手元に作った記録を、インターネット上のGitHubへアップロードします！

CLI

```
git push -u origin add-member-あなたのID
```

VS Code 「ブランチの発行」 または 「変更の同期」 ボタン

```
To https://github.com/guousuideo/git-lec.git
* [new branch]      add-member-taro-yamada -> add-member-taro-yamada
Branch 'add-member-taro-yamada' set up to track remote branch.
```

GitHubにあなたのブランチが届きました！ 🚀

Step 8. Pull Request (PR) を作成する

Pull Request (PR) とは？

「自分の変更を本番に取り込んでください！」という **チームへのお願い・提案** のことです。

GitHub画面での操作手順

1. リポジトリ (`git-lec`) のトップを開く
2. 上部の黄色いバー **「Compare & pull request」** をクリック
3. タイトルと本文を確認して **「Create pull request」** をクリック！

なぜPRを作るの？

- いきなり本番に合体させず、事前に内容をレビューできる！
- チーム全員に「何を追加したか」を共有できる！

Step 9. レビュー & Merge ! 🎉

🎉 その場で全員のPRをMerge（合体・承認）します！

GitHub上のPR画面が「**Merged (紫色のアイコン)**」に変わる瞬間を見届けましょう！

紫色の **Merged** マークがついたら、**あなたの一連の作業は大成功です！** 🎉

第5章 (24-27分)

草を確認しよう！

プロフィールに緑色のマス目をついたかな？

自分のプロフィールを見に行こう！

ブラウザで自分のプロフィールページを開いてみましょう：

<https://github.com/あなたのID>



今日のマス目に色がついて「草」が生えています！

「草が生える」＝あなたが世界に向けてオープンな開発活動を一步踏み出した証拠です！

🚩 もし草が生えていなかったら？「3大罨」

「PRはマージされたのに草が生えていない！」という場合のチェックリスト：

罨① メールアドレスの不一致（90%はこれ）

`git config user.email` のメアドと、GitHubに登録されているメアドが1文字でも違うと草になりません。

罨② まだマージされていない

ブランチにコミットしただけでは草になりません。
`main` にマージされて初めて反映されます。

罨③ 自分のFork（コピーリポジトリ）で作業してしまった

Fork先のコミットは、元のリポジトリにPRを出してマージされるまで草カウントされません。

罣①（メールアドレス不一致）の治し方

ターミナルで設定を修正し、GitHub側にもメールアドレスを追加すれば解決します！

1. 今の設定を確認

```
git config --global user.email
```

2. GitHubに登録しているメールアドレスに上書き設定

```
git config --global user.email "your-correct-email@example.com"
```

💡 GitHubの [Settings] ➡ [Emails] に、PC側で設定していたメールアドレスを追加登録することでも、過去のコミットが自分のものとして紐づけられて草が生えます！

第6章 (27-30分)

まとめ & AI時代のGit

AIがコマンドを打つ時代に、人間は何をするのか？

今日できるようになったこと 🎓

たった30分で、実務のエンジニアと同じスキルを習得しました！

- ☒ **バージョン管理の基礎**（ファイルの歴史を綺麗に記録・復元できる）
- ☒ **Gitの4領域モデル**（作業場 ➡ インデックス ➡ ローカル ➡ リモート）
- ☒ **GitHub Flowの実践**（clone ➡ branch ➡ commit ➡ push ➡ PR）
- ☒ **草を生やす仕組み**（メール認証・マージ条件）

番外編1：コンフリクト（衝突）と元に戻す方法

同じファイルの同じ行を2人が同時に編集すると**「コンフリクト（衝突）」**が起きます。

✂️ コンフリクトは怖くない

Gitが「どっちを採用する？」と親切に聞いてくれて
いる状態です。VS Codeならボタン1つで選べます。

⏮️ やり直す道具（revert）

間違えてマージした時は `git revert` で「打ち消す
コミット」を作れば安全に戻せます。

詳しくは 📄 `docs/05_extra_conflict_revert.md` と `conflict-playground/` をチェック！

番外編2：AIエージェント時代のGit操作

AIがGitを自律操作する時代へ

Claude Code や GitHub Copilot が、**自然言語の指示だけで一連のGit作業を自動完結**させます。

🗨️ 人間の指示

「自己紹介ファイルを追加して、ブランチを切ってPRまで作っておいて」

🤖 AIエージェントの自動実行

1. `git switch -c add-member-taro`
2. ファイル作成 & `git add .`
3. `git commit -m "docs: add..."`
4. `git push` & PR作成（完了！）

なぜAI時代に「4領域」と「diff」を学ぶのか？

⚠ AIも間違える

- パスワードや `.env` をコミットしてしまう
- 消してはいけないコードを消してしまう
- 勝手に `push --force` してしまう

🛡 人間の最後の責任

- AIが出した**差分（diff）**を自分の目で読む
- 「今どの領域に何があるか」を把握する
- 問題がないことを確認して**承認（Approve）**する

「AIに任せる範囲が増えるほど、何が起きたかを読み解く力が武器になる」

これからのおすすめアクション

1 もう1回PRを出してみる

`members/あなたのID.md` を編集して、趣味や特技を追記してPRを出してみよう！2回目は驚くほどスムーズです。

2 自分のリポジトリを作る

大学のレポートやプログラミングの学習記録をGitHubで管理してみよう！

3 Cypherのプロジェクトに参加する

コミュニティの開発プロジェクトで、仲間と一緒にチーム開発を実践しよう！

Happy Hacking! 🌿

GitHubの世界へようこそ！

学生AI/データサイエンスコミュニティ **Cypher**